BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

ML PROJECT-BASED LEARNING REPORT (WEEK 7)

1. Basic Information

Programs Covered:	**Program 1: Employee Management System** **Program 2: Library Management System**
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**
Project Date	**21/08/2026**

2. Introduction and Background

Week 7 covers two record-management tools: an Inventory Management System for products, suppliers, and stock, and an Online Shopping Cart for browsing a catalogue and managing a cart. Both separate core logic from input/output handling.

3. Objectives of the Project (Week 7)

Program 1 - Inventory Management System

- Manage products, suppliers, and stock levels via CRUD operations
- Track low-stock items against a fixed threshold
- Separate business logic from input/output handling

Program 2 - Online Shopping Cart

- Build a product catalogue and a shopping cart
- Support adding, editing, and removing cart items
- Calculate line totals and a cart grand total

4. Problem Statement / Driving Question

Program 1: How can stock, products, and suppliers be tracked accurately and flagged when low?

Program 2: How can a cart reliably track quantities and totals against a shared catalogue?

5. Methodology (Week 7 Activities)

Program 1 - Inventory Management System

- Modelled products, suppliers, and stock as separate dictionaries
- Built core functions (add_product, receive_stock, adjust_stock) with no I/O
- Built UI handler functions that call core functions and print results
- Added a low-stock report against a fixed threshold
- Tested add, update, delete, and stock adjustment flows

Program 2 - Online Shopping Cart

- Modelled a shared catalogue and a separate cart dictionary
- Built core functions (add_to_cart, update_cart_item, get_cart_items)
- Built UI handlers for adding, viewing, editing, and clearing the cart
- Computed per-line and grand totals from catalogue prices
- Tested adding new and existing products, and editing/removing cart items

6. Expected Outcomes / Deliverables (Week 7)

- Working Inventory Management System with stock tracking
- Working Online Shopping Cart with live totals
- Clear separation of core logic and UI code
- Low-stock alerts for inventory items
- Accurate cart totals across edits
- Future enhancement: persistent storage for both systems

7. Core Logic

Program 1 - Inventory Management System

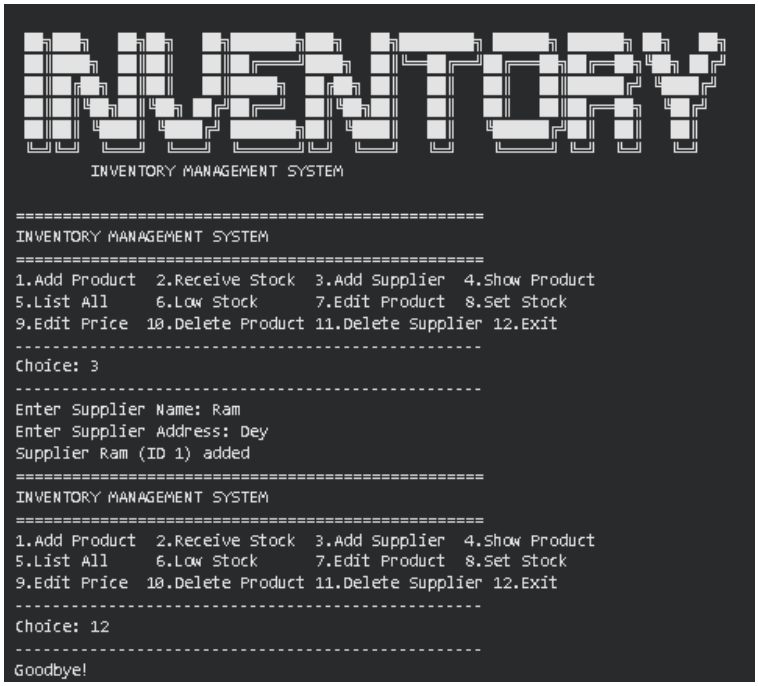
```
def receive_stock(pid, qty):
    if pid not in PRODUCTS: return False, "Product not found"
    if qty is None or qty <= 0: return False, "Qty must be >0"
    STOCK[pid] = STOCK.get(pid, 0) + qty
    return True, STOCK[pid]

def add_product(name, cat, price):
    if not name or not cat or price is None: return None
    pid = _next_pid()
    PRODUCTS[pid] = {"name": name, "category": cat, "price": float(price)}
    STOCK.setdefault(pid, 0)
    return pid

def check_low_stock(threshold=10):
    return [(pid, PRODUCTS[pid]["name"], STOCK.get(pid, 0))
            for pid in PRODUCTS if STOCK.get(pid, 0) < threshold]

def adjust_stock(pid, qty):
    if pid not in PRODUCTS: return False, "Not found"
    if qty is None or qty < 0: return False, "Qty >=0 required"
    STOCK[pid] = int(qty)
    return True, qty

def update_price(pid, price):
    if pid not in PRODUCTS or price is None: return False
    PRODUCTS[pid]["price"] = float(price)
    return True
```

Program 2 - Online Shopping Cart

```
def add_to_cart(pid, qty):
    if pid not in CATALOG: return False, "Not in catalog"
    if qty is None or qty <= 0: return False, "Qty must be >0"
    CART[pid] = CART.get(pid, 0) + int(qty)
    return True, f"{CATALOG[pid]['name']} x{qty} added"

def add_new_product(name, cat, price):
    if not name or not cat or price is None: return None
    pid = _next_id()
    CATALOG[pid] = {"name": name.strip(), "category": cat.strip(), "price": float(price)}
    return pid

def get_cart_items():
    items = []; total = 0
    for pid, qty in CART.items():
        if pid not in CATALOG: continue
        unit = CATALOG[pid]["price"]; line = qty * unit; total += line
        items.append((pid, CATALOG[pid]["name"], qty, unit, line))
    return items, total

def update_cart_item(pid, qty):
    if pid not in CART: return False, "Not in cart"
    if qty is None or qty < 0: return False, "Qty >=0 required"
    if qty == 0: del CART[pid]; return True, "Removed (qty 0)"
    CART[pid] = int(qty); return True, "Updated"

def remove_from_cart(pid):
    if pid not in CART: return False, "Not in cart"
    del CART[pid]; return True, "Removed"
```

8. Output Screenshot

Program 1 - Inventory Management System



Program 2 - Online Shopping Cart

9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core language for both console systems
Python Dictionaries	In-memory storage for products, suppliers, cart items
Core/UI Separation	Business logic functions kept separate from input/print handlers
Helper Functions	Shared ID-validation and prompt helpers to avoid repeated code
String Formatting (f-strings)	Aligned tabular display of products, stock, and cart

10. References

- Python Official Documentation: https://docs.python.org/3/
- Python Dictionaries Guide: https://docs.python.org/3/tutorial/datastructures.html
- Brainware University - Machine Learning Course Guidelines

11. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor:
